

VERITYLAKE

From first principles
to interview readiness

The complete project handbook

Understand the purpose. Follow every data stage. Explain the code, tools, decisions and failure boundaries.

28 learning chapters · 28 interview questions · Worked examples
Operational guide · Complete implementation reference

WEBSITE → LAKEHOUSE → CHECKED RELEASE → CITED ANSWER

Based on the local repository · 21 September 2026

Implementation descriptions are separated from measured runtime evidence. Illustrative examples are labeled throughout.

Contents

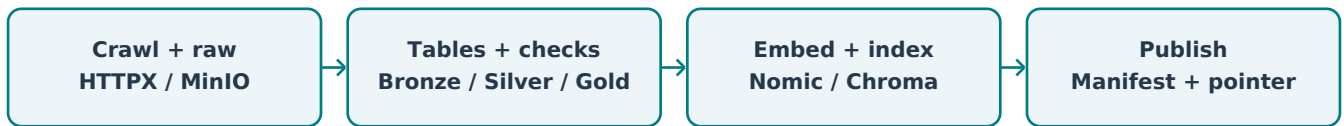
Select an entry to jump to its page. PDF bookmarks provide the same navigation.

Architecture at a glance	4
1. How to use this handbook	5
2. What problem does this project solve?	6
3. Foundations: the words you need first	7
4. The architecture as two connected workflows	8
5. Every major tool and why it exists	9
6. Stage zero: initialize a run	11
7. Scraping: turn pages into retained evidence	12
8. Bronze: the first structured table	13
9. Silver: clean, quarantine, deduplicate and enforce quality	14
10. Gold: make passages suitable for retrieval	15
11. Embeddings, Ollama and the vector index	16
12. Publication: when a candidate becomes visible	17
13. A question's complete journey	18
14. Citations, abstention and what correctness means	19
15. Follow one book through the entire system	20
16. Airflow: scheduling, retries and dependencies	21
17. Docker and local startup, from an empty machine	22
18. API behavior, authentication and the browser	23
19. Failure recovery, rollback and backups	24
20. Tests, evaluation and honest performance claims	25
21. Monitoring, delivery and cloud scope	26

22. Security boundaries and realistic limitations	27
23. Why these design choices, and how to extend them	28
24. Interview answers: purpose and architecture	29
25. Interview answers: implementation and difficult follow-ups	30
26. Demo plan, study exercises and final revision sheet	33
27. Code-reading lab: functions to follow	35
28. Evidence register and documentation boundaries	36
Implementation reference	37
1. What it is and why it exists	37
2. Concepts in plain language	38
3. System architecture	38
4. Why each technology is present	39
5. Website-to-release workflow	39
6. Question-to-answer workflow	41
7. API and UI	42
8. Complete application module map	42
9. Persistent data and identities	44
10. Docker services and startup order	44
11. Actual defaults	45
12. Run locally	45
13. Scripts and supporting files	47
14. Tests and evaluation	48
15. Monitoring, delivery, and cloud scope	48
16. Troubleshooting and limits	49
17. Suggested code reading order	50

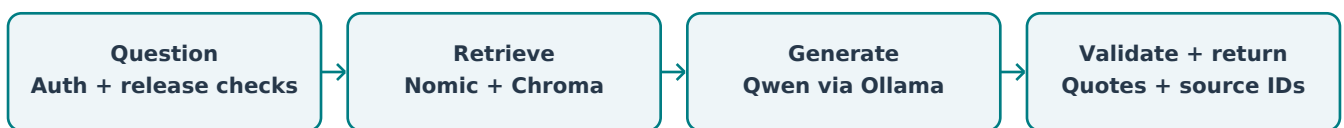
Architecture at a glance

BUILD AND PUBLISH / Airflow



Only a successful publication changes the selected release.

RETRIEVE AND ANSWER / FastAPI



MinIO: evidence and tables | PostgreSQL: Airflow state

DuckDB: deduplication and catalog | Chroma: retrieval text and vectors

Read the top lane when studying data engineering. Read the bottom lane when studying the application. Publication connects them: questions use a selected release rather than an unfinished candidate.

An end-to-end learning handbook for the repository, written for someone starting from zero.

Edition: 21 September 2026. Scope: the code in this project, its configured defaults, and the evidence recorded in the repository. Examples explicitly marked illustrative are teaching examples, not observed execution results. File paths are relative to the project root unless stated otherwise.

1. How to use this handbook

Read chapters 2–5 first to understand the purpose and vocabulary. Chapters 6–15 follow the data from a website into an answer. Chapters 16–23 explain operations, testing, security, design decisions, and extensions. Chapters 24–26 prepare you to explain and demonstrate the project in an interview. The implementation reference at the end provides module, script, configuration, and storage maps.

You do not need to memorize every dependency version. You should be able to explain what enters each stage, what changes, where the output lives, what is checked, and what happens on failure. Those five questions are the organizing principle of this book.

Read a chapter, draw its flow without looking, then open the named source file. Connect each concept to a function. Finally, answer that chapter's checkpoint aloud. This turns a document into an understanding of the implementation.

Evidence matters. The saved local setup report records 107 passed tests and two skipped modules. It also says full-stack startup, live Airflow publication, and a live answer were not yet verified when that report was written. This handbook does not establish a newer runtime status. A configured feature, a unit-tested behavior, and a measured live result are different claims.

2. What problem does this project solve?

VerityLake turns permitted website content into a checked, versioned knowledge collection, then answers questions using retrieved passages and source citations. The demonstration source is Books to Scrape, a book catalog designed for scraping exercises. Its prices and availability are demonstration facts, not a commercial inventory feed.

Imagine asking a chatbot for a book's listed price. A language model may know something about the title but has no inherent guarantee of knowing the page you collected. Even when you attach retrieved text, problems remain: a crawl might miss pages, duplicate content might dominate search, the dataset might be old, or an index might contain vectors created by a different embedding model.

The project addresses the whole chain. It saves the original evidence, creates structured tables, cleans and validates the documents, divides text into searchable chunks, builds an index, and publishes a release only after checks pass. The question-answering service reads the selected release. It can return an explicit abstention when it lacks sufficient valid evidence.

The distinguishing design is controlled publication. A newly built candidate does not automatically replace what users see. The application has a small active pointer that selects a validated release. If a new crawl or embedding stage fails, it does not move that pointer. The older release remains selected, although it can still become too old to serve under the freshness policy.

Potential adaptations include internal documentation, product manuals, and approved policy collections. These require suitable adapters, source permissions, access controls, and evaluation. This repository implements a small single-host portfolio platform. It does not establish enterprise scale, high availability, or universal factual correctness.

Checkpoint: Explain why a chatbot's answer quality depends on its ingestion pipeline, even when the language model itself works correctly.

3. Foundations: the words you need first

Data is the actual content: book titles, descriptions, prices, HTML, and text. Metadata describes that content: its URL, retrieval time, identifier, schema, and source location. A schema specifies the columns and allowed types in a record. For example, a title is a string, while a chunk ordinal is an integer.

A pipeline is a sequence of processing steps. An orchestrator decides when steps run, their order, and their retry behavior. In this project, Python functions perform the processing and Airflow orchestrates them. Running Airflow does not itself clean a document; it calls the code that does.

An API is an interface that software calls. A browser sends an HTTP request to FastAPI. Python sends another HTTP request to Ollama. JSON is the structured text format commonly used in those requests and responses. An endpoint is a particular API path such as `/ask` or `/api/embed`.

A database stores and retrieves information using defined operations. Different databases suit different jobs. PostgreSQL tracks Airflow's operational state. DuckDB runs local analytical SQL and stores a metadata catalog. Chroma searches vectors. MinIO stores objects, such as HTML files and table files, through an S3-compatible API.

A data lake stores many kinds of data as objects or files. A lakehouse adds table structure and versioned table management over that storage. VerityLake uses Delta tables on MinIO for this purpose. A table's files are stored in MinIO; Delta supplies table metadata and transaction-log semantics. These are cooperating layers.

ETL means extract, transform, load. ELT loads extracted data before later transformations. This implementation preserves raw content first and then produces multiple transformed layers, so describing the actual sequence is more informative than arguing over the acronym.

A batch processes a bounded group of records. A stream handles a continuing sequence of events. This project performs bounded batch crawls. It is not a Kafka streaming system, and it does not implement change-data capture from a transactional database.

A hash is a deterministic fingerprint of input bytes or text. The same input produces the same hash. Hashes help identify changes and build cache keys; they do not encrypt the original content. A UUID is an identifier used to distinguish a run. An ETag is a storage version token used here to detect whether an object changed before an update.

RAG means retrieval-augmented generation: find relevant stored text, provide it to a language model, and ask for an answer. An embedding is a numeric representation used to compare text. Inference means running an existing model. Training changes model parameters. This project performs inference and indexing; it does not train or fine-tune Qwen on the scraped books.

Checkpoint: Describe the difference between data, metadata, a database, an API, and a model in your own words.

4. The architecture as two connected workflows

The ingestion workflow prepares knowledge. The serving workflow answers questions. They share published metadata and the vector index, but a question does not trigger another web crawl.

INGESTION – controlled by Airflow

```
Website → Crawl → Raw objects → Bronze → Silver → Gold
                                     ↓
                               Embed → Chroma collection
                                     ↓
                               Checks → Release → Active pointer
```

SERVING – handled by FastAPI

```
Browser question → Authenticate → Resolve active release
    → Embed question → Search Chroma → Filter evidence
    → Qwen through Ollama → Validate citations → Response
```

Think of a library preparing a new edition of a reference collection. Raw HTML is the original material. Bronze is the initial transcription. Silver is the cleaned edition. Gold is the set of useful passages. Chroma is the semantic search index. The release manifest is the edition's inventory. The active pointer tells the librarian which edition to use. This analogy explains the roles; the actual implementation uses objects, tables, and HTTP calls.

Three different forms of versioning appear. Delta versions identify table snapshots. Run IDs group the artifacts of an ingestion attempt. The active pointer selects a release across those artifacts. Docker image digests identify application software, which is another independent version. Reproducing behavior requires considering both data and software/model versions.

The API reads the Chroma document text returned with vector matches. It does not run a DuckDB query over the entire lake for each question. Keeping the query path focused reduces the number of systems needed for retrieval, while the lake retains the data and provenance used to build the release.

Checkpoint: Draw both workflows and mark which component chooses the release users can query.

5. Every major tool and why it exists

Tool	How it works here	Why this project uses it
Python	Functions and classes implement ingestion, checks, adapters, CLI and API	One language connects the data and application layers
HTTPX	Sends bounded HTTP requests and reads responses	Access websites and Ollama with explicit timeouts
BeautifulSoup	Parses HTML into searchable elements	Extract book fields and discover links
MinIO / S3 API	Stores bytes under keys inside a bucket	Retain original evidence, tables, reports and releases
Boto3	Python client for S3 operations	Read and write objects, including conditional writes
PyArrow	Builds typed columnar tables from Python rows	Enforce the schema used for Delta writes
Parquet	Column-oriented file representation beneath Delta	Store structured table data efficiently
Delta Lake / delta-rs	Maintains a transaction log and versioned table snapshots	Reference exact table versions in releases
DuckDB	Embedded SQL engine over Arrow data and local files	Deterministic deduplication and metadata catalog creation
Airflow	Schedules a dependency graph and records task attempts	Retries, stage visibility and repeatable orchestration
PostgreSQL	Relational server used by Airflow	Persist orchestration state separately from the corpus
Ollama	Loads model weights and exposes local inference APIs	Run the encoder and answer model through HTTP
Nomic embedding model	Maps text into numeric vectors	Semantic retrieval for documents and questions
Chroma	Stores vectors, text and metadata; searches by cosine distance	Find likely relevant passages
Qwen3 4B	Generates text conditioned on the evidence and question	Write a readable answer in a structured response
FastAPI	Maps HTTP routes to Python handlers	Expose questions, catalog, health and metrics
Pydantic	Validates structured inputs and model outputs	Reject invalid settings and malformed requests/responses
Uvicorn	Runs the Python ASGI application server	Serve FastAPI over HTTP

Tool	How it works here	Why this project uses it
HTML / CSS / JavaScript	Browser markup, styling and request handling	Simple evidence UI without a frontend build pipeline
Docker Compose	Connects containers, volumes and startup dependencies	Reproduce the local multi-service environment
OpenLineage	Emits structured stage lifecycle events	Record which stages consume and produce datasets
Prometheus / Grafana	Scrape metrics and visualize them	Optional visibility into errors, latency and freshness
Pytest / Ruff	Execute tests and check Python code	Catch behavioral regressions and code issues
GitHub Actions	Runs declared automated workflows	CI, image release, deployment and live acceptance tooling
Terraform	Declares optional AWS storage and permission resources	Provide a cloud storage foundation without claiming a full cloud platform

Tools overlap in broad capability but have distinct jobs here. PostgreSQL is not the book store. DuckDB is not the vector server. Chroma is not the source archive. Ollama is not a model: it runs the Nomic and Qwen models. Docker is not a database: volumes preserve the databases' files.

The direct Python pins live in `requirements.txt`; package and CLI configuration live in `pyproject.toml`. Airflow is installed through its separate image. Pinned direct dependencies do not automatically mean every transitive dependency and upstream source commit is fully locked.

Checkpoint: For each tool, finish this sentence: "If we removed it, we would need another way to ____."

6. Stage zero: initialize a run

Source: `src/veritylake/pipeline.py`, `config.py`, and `dags/veritylake_dag.py`.

Before collecting data, `Pipeline.new_run()` establishes an identity for the work. A manual run normally gets a UUID. Airflow derives a stable ID from its DAG run ID so that retried tasks refer to the same candidate rather than accidentally creating unrelated candidates.

The run record stores its creation time, selected public configuration, a configuration fingerprint, and the active object's current ETag. Its path is `runs/<run_id>/run.json`. Credentials are not included in the public pipeline configuration.

The fingerprint prevents resuming a run with incompatible selected settings. Changing chunk size halfway through a candidate could otherwise combine outputs created under different assumptions. A retry with changed fingerprint is rejected and requires a new run. Not every operational setting is fingerprinted, so this is a deliberately scoped consistency check.

Capturing the active ETag prepares for publication much later. The publisher will say, in effect, “Switch the active pointer only if nobody changed it since I started.” This protects against a stale candidate replacing a newer release.

Each pipeline stage records status, timing, output metadata, or failure type under `runs/<run_id>/stages/`. A completed stage returns its recorded output on retry. Failed work can be attempted again. This provides useful idempotent behavior, but does not turn every network write into a global exactly-once transaction.

Checkpoint: Why are both a run ID and a configuration fingerprint needed?

7. Scraping: turn pages into retained evidence

Source: `crawler.py`, `text.py`, and `Pipeline.scrape()`.

The crawler starts at the configured website, reads robots rules, and follows allowed links. The books adapter recognizes product links and next-page links. Listing pages help discovery; selected product pages become documents. The default target is 60 selected documents, with at least 50 required and at most 180 visited URLs. “Pages fetched” and “documents accepted” are therefore different numbers.

HTTPX requests the page. BeautifulSoup parses its HTML. The books adapter reads the title, star rating, product-table fields, and description. It combines the relevant fields into text that later retrieval can use. It does not run browser JavaScript or require Selenium. The generic adapter removes common active/navigation elements and extracts a configured region, but is not a universal website parser.

The crawler enforces same-origin and path restrictions, checks public destination addresses, rejects unsafe URLs, checks redirects, caps response size, and uses bounded retries. Requests are paced with a minimum interval of 0.5 seconds, increased where robots rules require. These controls limit unwanted crawling and resource use. DNS checks alone are not a complete network egress security boundary.

For fetched pages, the pipeline retains response HTML bytes, extracted text, and page metadata. It also stores robots information, crawl errors, and the selected document list. If too few documents were collected, the stage fails after preserving diagnostics. A failed scrape should still leave useful evidence about why it failed.

<code>raw/<run_id>/pages/<doc_id>.html</code>	original response body
<code>raw/<run_id>/pages/<doc_id>.txt</code>	extracted text
<code>raw/<run_id>/pages/<doc_id>.json</code>	page metadata
<code>raw/<run_id>/documents.json</code>	selected structured rows
<code>raw/<run_id>/robots.json</code>	crawl policy evidence
<code>raw/<run_id>/errors.json</code>	crawl diagnostics

The document ID hashes the canonical URL. A content hash describes content. These answer different questions: “Is this the same source page?” and “Is its content the same?” A page can retain its identity while its description or price changes.

Retaining HTML helps investigate extraction bugs and audit the source. It makes reprocessing possible in principle, but the normal pipeline does not automatically implement every imaginable historical re-extraction workflow. Avoid claiming a feature merely because its necessary data has been saved.

Checkpoint: If only 30 valid product pages are found, what survives and why does the candidate stop?

8. Bronze: the first structured table

Source: `tables.py` and `Pipeline.bronze()`.

Bronze loads the selected extracted document rows into a Delta table. Raw HTML remains in object storage. Bronze is the first structured snapshot of extraction, with columns such as `doc_id`, `url`, `title`, `text`, `fetch_at`, `content_sha256`, `raw_key`, `etag`, `last_modified`, `status`, and `html_bytes`.

PyArrow constructs a typed table from the Python dictionaries. Delta writes table data and a transaction log beneath `bronze/<run_id>/documents`. The returned metadata includes the key, URI, Delta version, row count, and schema. Later stages read the specified version rather than assuming the latest possible state.

Parquet is the underlying columnar data format. Delta adds a table transaction log that identifies which files belong to a snapshot. MinIO holds those files and logs. PyArrow is the in-memory bridge used to construct and read the rows. These four names describe different layers of the same storage operation.

This implementation gives each run its own table paths and writes with overwrite mode. It is a per-run snapshot design. It does not implement a continuously merged customer table, slowly changing dimensions, or SQL MERGE-based incremental ingestion.

Why keep Bronze if Silver is cleaner? Bronze preserves the structured result before cleaning rules remove or normalize anything. When a good document disappears, compare extraction with cleaning. Without this boundary, diagnosis becomes much harder.

Checkpoint: Explain raw HTML versus Bronze, and Parquet versus Delta.

9. Silver: clean, quarantine, deduplicate and enforce quality

Source: `quality.py`, `text.py`, `tables.py`, and `sql/silver.sql`.

Silver begins by copying each document and normalizing its text and title. NFKC Unicode normalization makes some equivalent character representations consistent. Most control characters are removed; whitespace is collapsed; email patterns are redacted by default. The normalized text gets a new content hash.

Documents are rejected for missing titles or short text, selected instruction-like patterns, invalid timestamps, or timestamps that are too old or implausibly in the future. The minimum text length is 100 characters. Timestamps must include timezone information. The configured source age limit is 72 hours, with a five-minute tolerance for future timestamps in document preparation.

Rejected document identities and reasons go to `quarantine/<run_id>/silver.json`. Quarantine makes exclusion explainable. The raw source remains available separately. Redacting emails in normalized text does not erase emails from the retained raw HTML, and a few instruction patterns do not recognize every malicious prompt.

DuckDB then removes exact normalized-content duplicates. Its SQL partitions by `content_sha256`, uses `row_number()`, and chooses a deterministic winner ordered by URL and document ID. Determinism matters because a retry should not arbitrarily keep a different duplicate. This is exact-content deduplication, not fuzzy semantic duplicate detection.

Quality checks require enough surviving documents, unique document IDs, unique content hashes, an acceptable quarantine fraction, and nonempty output. The rejection budget is 20% by default. The quality report is saved before enforcement; if a check fails, publication cannot proceed through the normal stage chain.

Illustrative arithmetic: suppose Bronze has 60 rows. Four are quarantined and three additional duplicate rows are removed. Silver has 53 rows. The quarantine fraction is 4/60, or about 6.7%; duplicates are recorded separately and are not added to that fraction. With a minimum of 50 documents, these counts can pass if the other checks pass. If only 48 remain, the minimum-document check fails even with a small quarantine fraction.

The successful table lives at `silver/<run_id>/documents`. This is the normalized text against which later chunk offsets are checked. Always remember that offset provenance is relative to Silver text, not byte positions in the original HTML.

Checkpoint: Why can a candidate fail the minimum-document check even when its rejection budget passes?

10. Gold: make passages suitable for retrieval

Source: `text.py`, `quality.py`, and `Pipeline.gold()`.

Gold in this project is a table of searchable text chunks. In another project, Gold might mean business aggregates; the name describes a consumption-ready layer, not one fixed type of data.

The chunker finds words using non-whitespace matches, creates windows of 220 words, and overlaps adjacent windows by 35 words. The step is therefore 185 words. Overlap helps preserve information near boundaries, at the cost of repeated text and extra vectors. Words are not model tokens, so a 220-word window is not a guarantee of a particular token count.

Illustrative example: a 400-word document creates windows covering word indices [0,220) and [185,400). The second window retains 35 words from the end of the first. A 100-word document creates one chunk. The implementation stops once a window reaches the document end; it does not add an unnecessary trailing overlap-only chunk.

Each chunk stores its document ID, ordinal, URL, title, text, character start/end offsets, timestamp, raw key, content hash, and chunker version. Its ID incorporates document identity, normalized document content identity, chunking version, window size, overlap, and starting word index. Changing chunk settings changes chunk identity.

Gold checks that the chunk count is positive and no more than 1,500 by default, IDs are unique, every Silver document is covered, text sizes are bounded, and each chunk exactly equals the corresponding Silver substring. The offset interval is start-inclusive and end-exclusive, matching Python slicing.

The table is written to `gold/<run_id>/chunks`. It provides an auditable bridge between full documents and the passages used by retrieval. A citation can identify a document and chunk, while the raw key traces back to original acquisition.

Checkpoint: Explain why overlap helps and why exact offsets must refer to normalized text.

11. Embeddings, Ollama and the vector index

Source: `embeddings.py`, `vectors.py`, and `Pipeline.embed()`.

An embedding model converts a text string into a list of numbers. The numbers represent learned features that allow related texts to be nearby under a similarity measure. They are not a readable compressed answer, and a particular coordinate is generally not a human-defined attribute such as “price.”

Ollama is the model-running software. The configured encoder is `nomic-embed-text:v1.5`. The configured answer generator is `qwen3:4b`. Qwen writes answers; Nomic creates the retrieval vectors. Neither model is trained by this pipeline.

For document indexing, the pipeline embeds the title followed by a newline and the chunk text. The adapter prefixes that input with `search_document: .` For questions, it prefixes the question with `search_query: .` These are encoder-specific semantics. Replacing the encoder can require changing the adapter, rebuilding the index, and reevaluating retrieval, not just changing an environment variable.

The adapter resolves the model name and digest through Ollama's model listing. Identity includes the prefix version. A digest distinguishes actual model content from a tag that may move. The cache key combines this identity with the input text hash. Reusing the same text under the same identity can reuse its embedding across runs. The default embedding batch size is 16.

Vectors are checked for the expected count, consistent positive dimension, finite numeric values, and at least one nonzero entry. The model identity is checked again after encoding. A model change during indexing blocks the candidate rather than mixing incompatible vectors.

Chroma stores a collection named `vl_<run_id>`. It receives explicit vectors, chunk text, and source metadata; its embedding function is disabled because VerityLake supplies the vectors. It configures cosine space for its HNSW index. HNSW is an approximate nearest-neighbor indexing approach that avoids exhaustively comparing every vector in larger collections. Approximation is another reason to evaluate retrieval rather than assume perfection.

Cosine similarity compares vector direction: $\text{dot}(a,b)$ divided by the product of their lengths. Cosine distance is commonly 1 minus that similarity in this setting; smaller distances indicate closer vectors. The project's retrieval threshold is 0.65. A distance is not a probability that a passage is true or answers the question.

After upserts, the pipeline compares collection count with Gold row count and checks whether up to five chunks can be found among their own top matches. This self-retrieval check detects some index problems. It does not measure user-question accuracy, and equal counts alone do not prove every stored record is correct.

Why preserve both Gold and Chroma? Gold records the prepared dataset and source offsets. Chroma is the serving index derived from it. Keeping them distinct supports audit and rebuilding, although a rebuild still needs compatible model artifacts and code.

Checkpoint: Why is an unchanged vector dimension insufficient to prove compatibility between two embedding models?

12. Publication: when a candidate becomes visible

Source: `pipeline.py`, `publication.py`, and `storage.py`.

A release manifest ties together the run ID, table versions, quality reports, metadata catalog, vector collection, vector count, embedding identity, source information, configuration fingerprint, and Git SHA. It is stored at `catalog/releases/<run_id>.json`. A DuckDB metadata file contains dataset, column, and check records; it is not a second complete copy of the corpus.

The publisher validates release evidence, checks vector count and encoder identity, and prevents changing an existing manifest through the application. It records a publication intent before attempting to move `catalog/active.json`. An intent records an attempted operation; it does not prove that the operation succeeded.

The active pointer is the visibility boundary. Updating it uses compare-and-swap. With an existing pointer, S3 receives an `IfMatch` condition containing the expected ETag. For initial creation, `IfNoneMatch`: * requires that the object not already exist. The local store adapter uses its own local coordination rather than relying on a remote S3 service.

Illustrative race: candidates A and B both start while release R is active with ETag E. B finishes and switches the pointer, which now has a different ETag. A later tries to publish using E. Its conditional update fails. A must not blindly retry with a fresh ETag, because that would discard the original protection against overwriting newer work.

If the pointer switch succeeds but task acknowledgement fails, a retry can recognize that the same run is already active. If an earlier processing stage fails, the active pointer is unchanged. A failed candidate may leave retained artifacts requiring future cleanup; this repository does not implement reference-aware garbage collection.

This mechanism is not a distributed transaction across MinIO, Delta and Chroma. It provides a controlled final selection after checks. Administrative deletion or mutation can still damage a release. Application-level immutability is not storage-enforced object lock or cryptographic verification of every referenced file.

Checkpoint: Explain the exact moment users begin seeing a new dataset and what stops a stale writer.

13. A question's complete journey

Source: `api.py`, `rag.py`, `embeddings.py`, `vectors.py`, and `ui/app.js`.

The browser sends `POST /ask` with JSON containing the question and `top_k`, and puts the shared API key in the `X-API-Key` header. Pydantic checks the request. The question must be nonblank and no longer than 1,000 characters. `top_k` must be 1–10 and defaults to four.

FastAPI authenticates the key and acquires the request budget. Defaults allow 30 accepted answer requests per minute and two concurrent requests in the API process. These are process-local controls. Adding multiple workers would multiply independent counters rather than create a shared global limit.

`RAGService.checked_release()` loads the active release, validates its manifest, compares the installed encoder identity with the recorded identity, and checks freshness. The age check uses the run's creation timestamp. Publication time does not reset the age of an old run.

Ollama embeds the question using the query prefix. Chroma searches the selected collection and returns candidate text, metadata and distance. The service removes candidates beyond distance 0.65. It truncates each evidence text to at most 3,500 characters and keeps an overall 15,000-character budget. These character limits do not prove the entire prompt fits every model's token budget.

Surviving passages receive local source labels such as S1 and S2. The generator sees the question and these evidence records, with instructions to treat evidence as untrusted data and answer only from it. It is asked for structured JSON containing an answer, citations, and an abstain flag. The default request uses temperature zero, an 8,192-token context setting, a 700-token output setting, and thinking disabled.

The application validates the returned JSON and its citations. If valid, it returns the answer, sources, selected run ID, and model names. If no evidence survived, the model abstained, or citation checks failed, the response records the corresponding abstention reason. Dependency failures can produce HTTP 503 rather than pretending that an infrastructure outage is simply an unanswerable question.

The UI displays answers and quotes using `textContent`, which avoids interpreting answer text as HTML. It keeps the entered API key in page memory and restricts source links to HTTP(S). The normal query path neither executes model-generated commands nor performs another web crawl.

Checkpoint: Explain the difference between an unanswered question caused by missing evidence and one caused by a broken model service.

14. Citations, abstention and what correctness means

A generated answer can look plausible while being wrong. VerityLake introduces mechanical checks that are useful but limited. Citation source IDs must refer to supplied evidence; inline IDs and citation IDs must match; every quoted passage must be an exact contiguous substring of the supplied source text.

Illustrative valid provenance: the evidence contains “Price: £12.00” and the answer cites that exact text from S1. The check establishes that the quote was present. It does not independently prove that S1 concerns the requested edition, that the original website is accurate, or that every other sentence in the answer follows from the quote.

The prompt asks for every factual statement to have evidence. The code checks source-ID sets and quote membership; it does not implement a sentence-by-sentence logical entailment verifier. This is an important interview distinction: prompting describes desired behavior, while validation determines what is actually enforced.

Abstention is a successful application outcome when the evidence is insufficient. However, an assistant that abstains on every question is not useful. Evaluation must examine both correct abstentions and false abstentions on answerable questions.

Source offsets in the returned metadata identify the chunk's range in normalized Silver text. They are not computed as the exact location of the quoted phrase within that chunk. The quote itself is validated against the retrieved evidence text, which may have been truncated to the serving budget.

Temperature zero encourages more stable generation but does not promise byte-for-byte deterministic output across all model versions, hardware, and serving implementations. The generation model is configured by tag; the application's digest compatibility checks focus on the embedding model.

Checkpoint: Give an example of a real quote that could accompany a misleading answer and explain why human answer evaluation is still needed.

15. Follow one book through the entire system

The following book and values are illustrative. They demonstrate shapes and transformations rather than reproduce a live crawl. Suppose a source page for “Learning Gardens” includes a listed price of £12.00, a description, and enough text to pass the length requirement.

First, the crawler fetches its allowed URL and the adapter extracts the title, fields, and description. HTML is saved under the run's raw prefix. The URL produces the document identity, while response bytes produce acquisition content metadata. Bronze records the extraction and its raw-object reference.

Silver normalizes the text and redacts any matching email. If the title is present, timestamps are acceptable, content is long enough, and no configured instruction pattern matches, the document survives preparation. Its normalized text hash participates in exact-content deduplication. A second URL with identical normalized text would compete for the deterministic retained row.

Suppose the surviving normalized document is 400 words. Gold creates two overlapping chunks. Both retain the book URL and document ID, but each has a separate chunk ID and character range. Gold checks that slicing the Silver document with each range produces the corresponding chunk text exactly.

For each chunk, the embedding input contains its title and text. The cache looks for that input under the current encoder identity. Missing vectors are produced through Ollama. Chroma stores the chunks and vectors in the candidate collection. The pipeline validates counts and self-retrieval, then creates the release manifest and attempts the conditional active-pointer update.

Now a user asks, “What is the listed price of Learning Gardens?” The question is embedded and used to search the active collection. If the price passage survives retrieval filtering, Qwen receives it and can answer with an S1 citation. The validator checks that the source label exists and the quote was actually present. If the passage was not retrieved, the correct behavior is abstention rather than inventing a price.

Traceability runs backward too: response → run ID → release manifest → collection and Gold snapshot → chunk/document ID → Silver document → raw key → original retained HTML. Some of this trace is assembled from stored metadata rather than presented as one automatic UI action.

Now imagine the price changes tomorrow. A new ingestion creates a new run and dataset. Unchanged embedding inputs may hit the cache; changed inputs are re-encoded. The old release remains retained. Once checks pass and publication succeeds, new questions select the new release. This is refreshed batch ingestion with embedding reuse, not a per-record streaming update system.

Checkpoint: Describe which identities change when the page's text changes but its URL stays the same.

16. Airflow: scheduling, retries and dependencies

Source: `dags/veritylake_dag.py` and the Airflow services in `compose.yaml`.

A DAG is a directed acyclic graph: steps have dependencies and do not loop back into themselves. This project's graph is a straight chain: initialize, scrape, Bronze, Silver, Gold, embed, publish. A stage begins only after its prerequisite succeeds.

The scheduler decides which tasks are ready. The DAG processor discovers and parses DAG definitions. The Airflow API service supports its UI/API and task execution interface. LocalExecutor runs tasks on the same host environment. PostgreSQL stores the orchestration metadata, while task outputs are persisted in the lake.

The DAG passes only the run ID through XCom, Airflow's small task-message mechanism. Passing the whole corpus through orchestration metadata would duplicate large data and burden the scheduler database. Each task reconstructs the pipeline and reads the persisted results it needs.

The DAG is unpaused with an `@once` schedule. It runs once automatically under the configured conditions; it is not a daily refresh job. `catchup=False` prevents historical schedule backfill. Operators can trigger additional runs. Keeping data within the 72-hour freshness limit requires a refresh policy beyond the initial automatic run.

The limits are one active DAG run and one active task, two retries with 30 seconds between attempts, a 12-minute task timeout, and a 20-minute DAG timeout. One active run reduces normal Airflow overlap, while conditional publication still matters because manual tools or other writers can exist.

Retries help transient errors, such as a temporary network failure. They cannot repair a wrong selector, a permanently missing model, or incompatible configuration. Inspect the first failing stage and its diagnostics before repeatedly triggering new runs.

The DAG avoids network/model/database work at parse time. Parsing should declare tasks, not start crawling whenever Airflow loads the file. This separation is important for predictable scheduler behavior.

Checkpoint: Why are a stage's business logic and Airflow's scheduling logic kept separate?

17. Docker and local startup, from an empty machine

Source: `compose.yaml`, `Dockerfiles`, `Makefile`, and `scripts/init_*.py`.

A Docker image packages software and dependencies. A container is a running instance of an image. A named volume retains mutable files independently of a container's lifecycle. A Compose network lets services find each other by service name, such as `ollama` or `minio`.

On the host, `localhost` means your machine. Inside the API container, `localhost` means that container. Therefore the application uses `http://ollama:11434`, not host `localhost`, to reach its model server. Chroma and Ollama have no published host ports in the default stack; their callers are other containers.

Bootstrap generates a private `.env` file with separate credentials. Compose interpolates these values into an explicit set of service settings. A value appearing in `.env` is not automatically forwarded into every container; check the service environment configuration before assuming a new setting applies.

MinIO initialization creates the bucket and separate reader/writer credentials. Model initialization waits for dependencies, downloads both models, and warms the encoder. Airflow initialization prepares authentication and migrates its metadata database. The scheduler and application services depend on the required initialization states.

Initial startup can take much longer than a warm run: image downloads, source builds, Python packages, model downloads, database initialization, and the first crawl all contribute. CPU-only inference is configured by default; an optional GPU overlay exists. GPU memory requirements depend on model representation, context length and runtime settings, so a fixed VRAM guarantee cannot be inferred from "4B." Measure the actual chosen configuration. The repository's planning guidance is four CPU cores, 16 GB RAM, and 20 GB free disk; it is not a measured minimum.

Run these commands from the `veritylake` directory:

```
python3 scripts/bootstrap.py
docker compose config --quiet
docker compose up --build -d
docker compose ps -a
docker compose logs --tail=100 model-init airflow-scheduler api
```

Initialization containers can finish successfully and show an exited state. That is different from a long-running service repeatedly crashing. Read exit codes and logs in context rather than expecting every service to remain running forever.

After a release has been published, verify readiness and an actual useful answer:

```
curl --fail http://127.0.0.1:8000/readyz
python3 scripts/ask.py "What is the listed price of A Light in the Attic?"
python3 scripts/smoke.py
docker compose run --rm tools verify-release
```

Open the evidence UI at `http://localhost:8000`, Airflow at `http://localhost:8080`, and the MinIO console at `http://localhost:9001`. Read credentials locally from `.env`; do not paste them into public interview materials. `docker compose down` stops/removes containers while retaining named volumes. Adding `--volumes` changes that behavior and can delete persisted data.

Checkpoint: Why can a healthy container and a loaded web page coexist with an application that is not ready to answer?

18. API behavior, authentication and the browser

Interface	Meaning	What a caller should infer
GET /healthz	API process is alive	Does not establish usable data or models
GET /readyz	Release/model/index readiness checks	Useful dependency readiness, not a full answer-quality test
GET /catalog	Active release and checks	Requires the API key
POST /ask	Validated question-answer request	Preferred over placing question text in a URL
GET /ask	Query-parameter alternative	Question may appear in URL logs/history
GET /metrics	Prometheus metrics	Local operational interface
GET /docs	Interactive API documentation	Protected operations still require authentication

HTTP 401 means the supplied authentication is not valid. HTTP 422 means the request violates the input schema. HTTP 429 means the request budget is exhausted. HTTP 503 means a dependency or selected release is unusable. An abstained response is an application-level outcome and need not be an HTTP error.

Readiness checks encoder identity, dataset age, collection count, and generation-model availability. A real smoke test goes further by asking a known answerable question and requiring a cited, non-abstained answer. Neither a successful TCP connection nor a 200 response from liveness substitutes for that check.

The UI is ordinary browser JavaScript. It reads inputs, sends requests, and renders the resulting answer, sources and catalog. There is no React build chain to debug. The simplicity makes the request flow easy to inspect in browser developer tools, but it does not provide user accounts or tenant permissions.

Checkpoint: Explain how you would distinguish wrong credentials, invalid input, overloaded inference, and stale data from HTTP behavior.

19. Failure recovery, rollback and backups

Failures are expected in systems with networks, model servers and persistent storage. The important questions are whether users see incomplete data, whether failures are diagnosable, and whether recovery can preserve a known state.

Failure	Expected boundary	Investigation or recovery
Too few crawled documents	Scrape fails before later stages	Inspect robots, fetch errors and URL selection
Excessive bad/duplicate records	Silver gate can fail	Inspect quarantine and row-count report
Wrong chunk offsets	Gold gate fails	Compare chunk slices with normalized Silver text
Invalid or mismatched embeddings	Embed stage fails	Inspect encoder response, cache identity and dimensions
Another publisher wins	Conditional pointer update conflicts	Inspect active/history; create a suitable new run
Ollama unavailable	Readiness/answer dependency failure	Restore the service and check model availability
Existing release too old	Serving freshness check fails	Ingest fresh source data
Generated quote is invented	Citation validation abstains	Inspect retrieval and output; do not bypass checks blindly

Rollback changes the pointer to a retained release after verifying the target's vector count and embedding identity. It does not restore missing vectors, switch model weights automatically, or make old data fresh. If the previous release uses a different encoder, matching model availability must be restored before switching.

```
make history
make rollback RUN_ID=<retained-run-id>
```

The backup script stops the stack and archives six persistent volumes with checksums. Stopping writers helps obtain a consistent single-host filesystem snapshot across these stores, at the cost of downtime. The script leaves the stack stopped. The `.env` credentials must be preserved separately; optional dashboard/metrics volumes are outside the six-volume set.

Restore verifies checksums and targets absent destination volumes. Rehearse using a fresh project name and matching credentials. A backup file proves only that an archive was created; a restore rehearsal demonstrates that it can actually recover the services and data you need.

Checkpoint: Why is switching a release pointer different from restoring a backup?

20. Tests, evaluation and honest performance claims

Tests answer specific questions. A unit test with a fake model can verify what happens when the model returns an invalid quote. It cannot establish the real model's answer quality or memory use. A local Chroma integration test exercises real library behavior, but does not alone prove the Compose service network works.

The recorded local setup result is 107 passes and two skipped modules, with two deprecation warnings. The skipped areas were actual Airflow import and the opt-in live service stack. Native local Delta, DuckDB and Chroma tests ran; S3 testing used Moto simulation. This is useful scoped evidence, not a completed live demonstration. Historical packaging reports contain a different older pass/skip count; do not combine the two as one execution.

There are four evaluation levels: pipeline correctness, index integrity, retrieval quality, and answer quality. Pipeline correctness covers gates, retries and conflicts. Index integrity covers vector count and self-retrieval. Retrieval quality asks whether independently relevant sources appear. Answer quality asks whether the response is correct, supported and useful.

Recall@k measures the fraction of labeled relevant URLs found in the top k results. If a question has two expected URLs and one is retrieved, recall is 1/2. Reciprocal rank is 1 divided by the rank of the first relevant result, or zero if none appears. Mean reciprocal rank averages this across questions. The evaluator deduplicates URLs for ranking because several chunks can come from one document.

The supplied retrieval fixture has four smoke cases. It queries retrieval directly, not the entire filtered generation-and-citation path. It is too small to establish broad accuracy claims. Expand it with held-out questions, absent facts, near-duplicate titles, multiple-source requests and adversarial evidence, then review answers independently.

Measure correct-answer rate, supported claims, useful-answer rate, citation validity, appropriate abstention, and false abstention separately. A genuine source quote and a high retrieval score are not substitutes for human factual assessment.

For performance, distinguish cold setup, warm services with uncached text, and warm services with cached embeddings. Report hardware, model identities, document/chunk counts, cache statistics and stage timings. CLI pipeline timing differs from Airflow trigger-to-completion timing, which includes scheduling and observation overhead.

The repository defines a five-minute warm pipeline target. A target is not an observed result. Use actual benchmark outputs and never invent latency, accuracy, throughput, savings or scale figures for a résumé.

```
make check
python3 scripts/smoke.py
make evaluate
make benchmark
make benchmark-airflow
```

The last two commands perform actual ingestion work; they are not read-only report viewers. `make check` uses the active Python environment, so install development dependencies in the project's isolated environment first.

Checkpoint: Explain why 107 passing tests, perfect self-retrieval, and a low average latency would still not prove answers are factually correct.

21. Monitoring, delivery and cloud scope

Logs describe individual events. Metrics summarize numeric behavior over time. Lineage describes relationships between processing stages and datasets. A release manifest describes one selected candidate's artifacts and evidence. All four help operations, but they answer different questions.

Pipeline stage records retain timing and failures. OpenLineage events record START, COMPLETE or FAIL with input/output identifiers and attempt information. They are saved durably to object storage; optional forwarding is best-effort. There is no requirement for a running Marquez server in the default stack.

Prometheus can scrape API request counts, latency histograms, answer/abstention counts, readiness observations, dataset creation time and active chunk count. Grafana displays those metrics. A metric describing the most recent readiness probe can be old if probes stop; understand the measurement before interpreting a dashboard.

`make observe` starts the optional observability services. The repository includes alert rules and a k6 workload. Their existence does not prove alerts reach an operator or that a load test has passed on this machine. Operational evidence requires actual execution and review.

CI checks source behavior, dependency/security conditions and selected integration contracts. Release automation builds images, attaches software inventory/provenance, scans image digests and signs successful releases. An SBOM lists software components; a signature supports artifact identity and provenance, not absence of vulnerabilities or correctness.

The deployment workflow updates an API image on an already provisioned host and checks readiness, with a return to the prior image on failure. It does not upgrade the full set of stateful stores or orchestrators. The live acceptance workflow is opt-in tooling, not proof that live acceptance has already run.

Terraform defines an optional AWS S3 bucket, DynamoDB coordination resource and IAM policies, with optional attachment to existing roles. It does not provision a complete network, compute fleet, model service, or identity provider. The local project does not require a cloud deployment. Do not describe Terraform files as a deployed AWS platform unless deployment evidence exists.

Checkpoint: What different evidence would you look for to prove an image was built, scanned, signed, deployed, and successfully used?

22. Security boundaries and realistic limitations

The intended default boundary is a local machine with loopback-exposed interfaces and internal model/vector services. The API uses one shared key. MinIO initialization gives the pipeline a writer role and the API catalog-reader access. The API needs to select published metadata; it does not need broad lake-writing privileges.

Crawler URL checks reduce the chance of reaching private services through untrusted links. Response-size, fetch-count and timeout limits bound work. Quarantine rules catch selected instruction patterns, and the generator prompt separates evidence from instructions. None of these establishes complete protection against every malicious input.

The UI's safe text rendering addresses HTML injection in displayed content. It does not replace authentication, TLS or tenant isolation. The default arrangement has no per-user document permissions or enterprise identity integration. The rate limiter is in one process, so horizontal scaling requires shared enforcement if a global limit is desired.

Retained raw content can contain material redacted from Silver. Retention and access policy must therefore cover the raw lake as well as the clean tables. A data-deletion requirement would need coordinated handling of raw objects, tables, caches, vector collections, releases and backups. That workflow is not implemented here.

Current storage and pipeline bounds suit small batches loaded into Python memory. There is no distributed compute engine, change feed, automatic reference-aware cleanup, multi-region failover, or full semantic verifier. The MinIO source-build choice and maintenance assumptions are documented in docs/DECISIONS.md and THIRD_PARTY_NOTICES.md; production selection requires a separate current maintenance review.

Checkpoint: Identify three implemented protections and three guarantees the project does not make.

23. Why these design choices, and how to extend them

Why DuckDB instead of Spark? The configured corpus is small, and local SQL over Arrow data provides the needed deduplication without a distributed cluster. A Spark migration should follow measured scale requirements, partitioning design and operational capability, not a desire to list more tools.

Why Delta instead of only JSON? JSON is convenient for raw diagnostics and manifests, while Delta provides typed structured snapshots and versions. JSON still exists where small readable records make sense. The tradeoff is added table-format dependencies and coordination concerns.

Why a separate collection per run? It keeps a candidate isolated from the active index and makes rollback selection simple. The cost is repeated storage and the need for a careful retention strategy. Collection-per-run does not remove the need to validate model identity or preserve source artifacts.

Why local models? They demonstrate a workflow where model inference runs locally after installation and can avoid transmitting retrieved passages to a hosted inference service. The costs include downloads, memory use, CPU/GPU capacity and operating the model server. Website ingestion still needs network access; local inference does not make the entire platform offline.

Why no reranker or hybrid search? The implemented retrieval baseline is cosine vector search with a distance filter. Exact title/identifier matching and semantic retrieval may benefit from lexical search or reranking, but these are extensions to evaluate. Adding them without measurements can increase latency while failing to solve the actual retrieval error.

To add a website, first establish permission and URL policy, then implement extraction and fixtures. Inspect raw outputs before adjusting quality thresholds. Keep source-specific behavior in adapters and verify that chunking captures the facts needed by intended questions. A generic selector change is insufficient for sites whose content requires JavaScript execution.

To change the embedding model, implement its required input semantics, version identity, rebuild vectors into a new candidate collection, and evaluate held-out retrieval questions. Keep model artifacts needed by retained releases if rollback matters. To change the generator, test structured output, citations, abstention, context behavior and latency; document its identity for reproducibility.

To scale ingestion, first measure memory and stage time, then consider partitioned processing, incremental source detection, bounded concurrency and a catalog/retention strategy. To scale serving, consider shared rate limits, authentication, model scheduling and consistent release selection. To add production reliability, address maintained dependencies, restore rehearsals, TLS, secret management, alert delivery and upgrade ownership.

Checkpoint: Pick one extension and explain what new failure modes it introduces, not just its benefits.

24. Interview answers: purpose and architecture

Q1. Tell me about this project in 30 seconds.

“VerityLake is a local data-to-RAG platform. It collects permitted website content, keeps the original evidence, builds Bronze/Silver/Gold Delta tables, validates the data, and creates an embedding index. A release becomes visible only after checks and a conditional pointer update. A FastAPI application uses that release to answer questions through local models and validates source IDs and exact quotes. The project emphasizes data provenance and failure handling.”

Q2. Give me a two-minute technical explanation.

Start with the book-catalog use case. Explain raw preservation and the three table layers. Then explain Nomic embeddings through Ollama, a separate Chroma collection for each run, and a manifest binding data versions to encoder identity. Describe conditional publication and an example of a failed candidate leaving the previous selection intact. Finish with the question path, citation validation, and the distinction between implemented checks and unverified live performance. This order connects every tool to a reason.

Q3. What is the main engineering contribution?

The coherent publication and provenance workflow is the strongest design point. Retrieval depends on compatible data, chunks and vectors. The release selects these as a checked candidate rather than exposing partial updates. Support the answer by opening `pipeline.py`, `publication.py`, and tests for stale-writer conflicts.

Q4. Is this data engineering, machine learning or backend development?

It combines data ingestion, transformation, quality and lineage with model inference and an API. The data engineering work produces the corpus and index; the AI layer performs embedding and generation; the backend serves authenticated requests. There is no model-training pipeline.

Q5. Why not send the whole website to Qwen?

The complete corpus may exceed useful context and introduce irrelevant material. Retrieval selects likely relevant chunks under a bounded context budget. It also supplies source metadata for evidence presentation. Retrieval can miss the needed passage, so it needs independent evaluation.

Q6. Is Ollama a model? Where are the two models used?

Ollama is the inference server/runtime. Nomic embeds documents and questions. Qwen generates answers from retrieved evidence. The project calls different Ollama endpoints for these responsibilities.

Q7. Does RAG train the model on the documents?

No. It builds a searchable external index and supplies retrieved text at inference time. Model weights are unchanged. Updating the dataset generally requires ingestion and indexing, not fine-tuning Qwen.

Q8. Why so many stores?

MinIO retains objects and Delta files; DuckDB performs local analytical work and stores catalog metadata; Chroma serves vector retrieval; PostgreSQL supports Airflow. Each has a specific responsibility. For a simpler prototype some could be removed, but that would change the demonstrated capabilities and tradeoffs.

25. Interview answers: implementation and difficult follow-ups

Q9. What makes a pipeline retry safe?

A stable run ID, frozen selected configuration, stored completion records, deterministic transformations and conditional publication reduce duplicate or inconsistent effects. Completed stages reuse recorded outputs. Partial failures still need care; this is not a claim of exactly-once behavior across all external systems.

Q10. Explain your deduplication SQL.

Prepared rows are partitioned by normalized content hash. `row_number()` orders each group by URL and document ID. Keeping rank one gives one stable winner per exact-content group. Final ordering by document ID makes output deterministic. Near duplicates require a different method.

Q11. Why use hashes for different identities?

URL identity tracks a document, content identity tracks changes, chunk identity tracks a precise derived window, and configuration identity detects incompatible run settings. Reusing one hash for all these meanings would obscure what actually changed.

Q12. What if the embedding model changes but dimensions stay the same?

The coordinate space can change even with the same number of coordinates. Comparing new query vectors with old document vectors becomes unreliable. The project compares identity including model digest and prefix version, and blocks incompatible serving/publication.

Q13. Explain compare-and-swap without jargon.

“Write this new selection only if the existing selection is still the one I observed.” S3 conditions make that check part of the write. A separate read followed by an unconditional write would leave a race between the two operations.

Q14. Is publication fully atomic?

The final pointer update is conditional and is the visibility point. The many prior object and vector writes are not one cross-system transaction. A manifest or intent can exist without becoming active. Retained artifacts can be damaged by external administrative actions.

Q15. What happens if publication succeeds and the task crashes?

On retry, publication can detect that the same run is already active and acknowledge it. This covers the gap between a successful pointer write and recording task completion, without requiring another blind pointer switch.

Q16. What happens when new ingestion fails?

The selected release remains unchanged because the failing candidate has not reached a successful publication. That older release still has to pass serving checks. If it is stale or its dependencies are broken, retaining its selection does not guarantee availability.

Q17. What exactly is checked in an answer?

Structured output must parse, cited source IDs must exist and match inline IDs, and each quote must appear verbatim in supplied evidence. These establish citation provenance. They do not prove logical entailment of every generated claim or source truthfulness.

Q18. What is the difference between quality and freshness?

Quality includes structural checks, completeness, uniqueness, valid offsets and vector invariants. Freshness concerns age. A structurally valid dataset can become stale; a newly collected dataset can still fail quality checks.

Q19. How do you prevent prompt injection?

The project quarantines selected instruction patterns, treats retrieved evidence as untrusted in the prompt, and validates output citations. These are limited layers, not a complete defense. The model is not granted a tool execution path in this application.

Q20. What are your accuracy and latency numbers?

Use only recorded measurements with their scope. The saved test report is not a live accuracy or performance benchmark. The repository includes evaluation and benchmarking tools, but a five-minute target and four retrieval smoke cases do not justify broad numerical claims.

Q21. How would you debug a wrong answer?

Trace the response's run ID to the release. Check whether the source fact was ingested correctly, survived Silver, appeared in a usable chunk, was retrieved, survived the distance filter and truncation, and was represented correctly by generation. Separate extraction, retrieval and generation failures before changing prompts.

Q22. Why not expose the latest collection immediately?

It may still be incomplete or incompatible. Separate candidate construction from selection lets checks finish first. The active pointer directs queries to an explicitly published candidate, while per-run collections avoid mutating the previous selection during ordinary ingestion.

Q23. What does rollback require?

A retained valid manifest, compatible available encoder, and the expected vector collection/count. It changes selection, not missing storage. Freshness remains enforced after rollback.

Q24. How would you make this production-ready?

Begin with the intended workload and threat model. Add suitable identity and authorization, maintained storage, TLS and secret management, shared rate limits if scaling, representative evaluation, reference-aware retention, tested recovery, alert delivery and an upgrade strategy. Do not equate more containers with production readiness.

Q25. Why is this not a streaming pipeline?

It crawls a bounded batch into a new run and publishes a snapshot. The default DAG schedule is once. There is no event broker, continual change feed, or stream-processing engine. Repeated batch refresh is a valid design, but should be described accurately.

Q26. What does your cloud infrastructure actually provision?

Optional storage and permissions: S3, DynamoDB coordination and IAM policy resources. It does not deploy the entire application or prove any cloud resources currently exist. Deployment evidence must be shown separately.

Q27. What did you personally build or change?

Answer from your actual work history. Separate original repository implementation, your modifications, and features you have studied or verified. You can explain a design deeply without falsely claiming authorship. Keep a small list of your actual changes, the reason for each, and the test or observation that supports it.

Q28. What is the most important limitation?

Choose a concrete one relevant to the question: a single-host batch architecture, limited answer-quality evaluation, or provenance checks without full semantic verification. Explain its operational consequence and how you would measure an improvement. Avoid claiming there is one universal limitation for every use case.

26. Demo plan, study exercises and final revision sheet

A good demonstration follows evidence. First show the architecture and explain the intended question. Then inspect an actual run in Airflow. Show raw evidence, Bronze/Silver/Gold counts, quality reports, and the release selection. Ask a known answerable question and inspect its quoted source and run ID. Ask a deliberately absent fact and inspect whether abstention is appropriate. Present only outcomes actually observed.

Prepare an isolated environment before failure demonstrations. Useful scenarios include a quality-gate failure that leaves the active pointer unchanged, an unavailable model showing liveness/readiness differences, a stale publication conflict, and a retained-release rollback. Tests demonstrate some of these contracts; distinguish test fixtures from a recorded service-level demo.

A five-minute interview explanation can allocate one minute to the problem and architecture, two minutes to ingestion and publication, one minute to serving and evidence, and one minute to measured results and limitations. Do not spend most of the time naming tools without explaining what crosses their boundaries.

Practice exercise 1: Draw a table with one row per stage and columns for input, transformation, output, validation and failure. Fill it from memory, then compare it to Pipeline.

Practice exercise 2: Using invented counts, calculate Silver output after quarantine and duplicates. Evaluate the minimum-document and rejection-budget checks separately. Explain why deduplicated rows and quarantined rows mean different things.

Practice exercise 3: Chunk a 400-word document with 220-word windows and 35-word overlap. Explain word versus character offsets, then verify why the stored slice must match exactly.

Practice exercise 4: Draw two publishers racing from the same ETag. Identify which write fails and why ignoring that failure would defeat the protocol.

Practice exercise 5: Open `rag.py` and identify one rule enforced by Python and one rule expressed only in the model prompt. Explain the difference in assurance.

Practice exercise 6: Trace a hypothetical missing-price answer through acquisition, cleaning, chunking, retrieval, truncation and generation. Name an artifact or function you would inspect at every step.

Quick revision sheet

Remember	Explain it this way
Purpose	Checked website data becomes a published, queryable knowledge collection
Raw	Original evidence and acquisition diagnostics
Bronze	Initial structured extraction in a versioned table
Silver	Normalized, checked, deduplicated documents
Gold	Overlapping source-linked text chunks
Encoder	Nomic through Ollama; model-aware cached vectors
Index	A Chroma collection per run with cosine search
Publication	Checked manifest plus conditional active-pointer update
Generator	Qwen through Ollama using retrieved evidence
Answer validation	Consistent source IDs and exact quotes, not a truth proof
Orchestration	Airflow orders tasks, retries and records operational state
Evidence	Tests, service checks, retrieval evaluation and human review answer different questions

27. Code-reading lab: functions to follow

Start with `Settings` in `config.py`. Find bounds for document counts and chunk overlap, then inspect `public_pipeline_config()` to see which settings are fingerprinted. Compare this list with Compose's explicit environment mapping. This shows the difference between defining a setting and actually supplying it to a container.

Open `Pipeline.new_run()` and `_stage()` in `pipeline.py`. Follow where run and stage records are written. Then read `scrape()`, `bronze()`, `silver()`, `gold()`, `embed()` and `publish()` in order. Notice that stage results contain references and summaries, not only raw business rows.

Follow `prepare_documents()` into `normalize()`. Read the actual instruction-pattern expression rather than assuming an AI classifier exists. Follow deduplication into `sql/silver.sql`, and follow `chunk_document()` into Gold offset checks. These are the main deterministic data transformations.

Read `OllamaEmbedder.identity()` and `embed()`, then `EmbeddingCache.encode()`. Trace why title-plus-chunk is the cached input and how a prefix-version change affects identity. Read `ChromaIndex.write()` to see which metadata is stored with vectors and which collection settings are checked.

Read `Publications.publish()` and `_switch()` alongside `S3Store.compare_and_swap()`. Identify the manifest write, intent write, pointer write and retry recognition as separate operations. This is the most useful code path for a discussion of concurrency and failure boundaries.

Finally follow `create_app()` to `RAGService.ask()`, `OllamaGenerator.generate()` and `validate_citations()`. Compare `ready()` with `ask()`; do not assume every readiness probe is repeated identically on every answer request. Finish by opening the corresponding tests to see examples of expected failures.

28. Evidence register and documentation boundaries

This handbook was prepared from the local source and repository documentation on 21 September 2026. Its purpose is an accurate learning reference, not a certification of live startup. The saved setup report says 107 tests passed and two modules were skipped, and records that full-stack startup and a live answer were not yet verified at the time of that report. Current runtime state must be checked separately.

Primary local references are `src/veritylake/`, `dags/veritylake_dag.py`, `compose.yaml`, `scripts/`, `tests/`, `eval/`, `.github/workflows/`, and `infra/terraform/`. Supporting explanations are in `docs/ARCHITECTURE.md`, `DECISIONS.md`, `RUNBOOK.md`, `DEVOPS.md`, `EVALUATION.md`, `PORTFOLIO.md`, `SOURCES.md`, and `END_TO_END_GUIDE.md`.

The following implementation reference incorporates the existing codebase guide, including its module maps, defaults, artifact paths and troubleshooting material. Host-capacity observations and historical test counts there retain their original scope; they are not new measurements. Dependency behavior described here is tied to this repository's code and pins, not a claim about the latest upstream releases.

Implementation reference

The repository's end-to-end codebase guide, included for lookup. Its historical setup observations retain their original scope.

This guide describes the implementation in this repository, inspected on 2026-09-21. Commands run from the `veritylake` directory. Runtime results belong in `reports/runtime/`; the supplied packaging reports are historical evidence, not proof that this installation works.

1. What it is and why it exists

VerityLake is a local data platform that turns permitted website content into a versioned, searchable dataset and answers questions with cited evidence. Its example source is Books to Scrape, a public demonstration book catalog. Book prices and stock are example data.

The problem is that a chatbot can sound convincing even when its source pipeline is incomplete, stale, duplicated, or using incompatible embeddings. VerityLake makes publication an explicit decision: build a candidate, test its data and index, record its provenance, and only then make it visible to users.

For example, a user asks, “What is the listed price of A Light in the Attic?” The system retrieves book text from the selected release, asks a local language model to answer using that text, checks its quoted evidence, and returns the answer with source links and the release ID. If a later crawl fails, the earlier release stays selected. Its normal freshness checks still apply; retaining a release does not make old data fresh.

Useful applications of this design include controlled knowledge collections, internal documentation search, and auditable AI demonstrations. Those domains would need their own ingestion adapters, permissions, evaluations, and operating controls. The current implementation is a single-host portfolio platform, not a finished multi-tenant enterprise service.

2. Concepts in plain language

Concept	Meaning here
RAG	Retrieve relevant stored text before asking a language model to generate an answer.
Data lake	Files and metadata stored in the MinIO bucket, using an S3-compatible API.
Lakehouse	Structured Delta tables in that lake, including schemas and versioned transaction logs.
Bronze / Silver / Gold	Extracted documents / cleaned documents / searchable text chunks.
Embedding	A numeric representation of text used to compare semantic similarity.
Vector index	Chroma collections containing embeddings, text, and source metadata.
Release	A manifest tying table versions, model identity, quality evidence, and one vector collection together.
Active pointer	catalog/active.json, the small object selecting the visible release.
Compare-and-swap	Update the pointer only if its stored version still matches the expected version.
Lineage	Records showing which stage consumed and produced which datasets.
Abstention	An explicit refusal to answer because adequate validated evidence is unavailable.

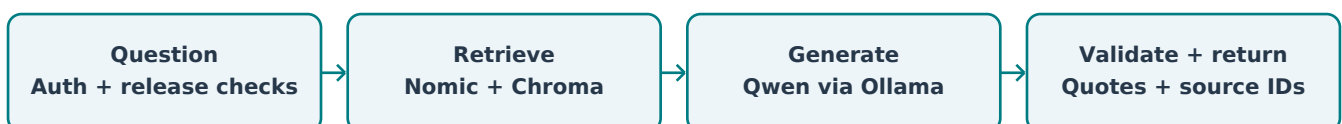
3. System architecture

BUILD AND PUBLISH / Airflow



Only a successful publication changes the selected release.

RETRIEVE AND ANSWER / FastAPI



MinIO: evidence and tables | PostgreSQL: Airflow state

DuckDB: deduplication and catalog | Chroma: retrieval text and vectors

Airflow controls ingestion. FastAPI serves questions. Both use the same Python package, but serving reads the published catalog and vector collection instead of executing the ingestion pipeline for every question. PostgreSQL stores Airflow metadata; it does not store the book corpus.

4. Why each technology is present

Technology	Actual responsibility and reason
Python	Shared pipeline, adapters, validation, API, and command-line implementation.
HTTPX	Bounded HTTP access for crawling, model inference, and optional lineage delivery.
BeautifulSoup	Extract book fields and links from HTML without a browser engine.
MinIO / S3	Persist raw evidence, tables, reports, manifests, and cache independently of application containers.
PyArrow	Explicit typed schemas and conversion between Python rows and columnar tables.
Delta Lake	Store Parquet-backed snapshots with exact versions that releases can reference.
DuckDB	Deduplicate a small batch using SQL and build a queryable metadata catalog.
Airflow LocalExecutor	Stage dependencies, retries, history, and operational visibility on one machine.
Ollama	Run the embedding and generation models locally after downloading them.
Chroma	Search precomputed embeddings using cosine distance.
FastAPI / Pydantic	Validate requests and responses and expose the serving interface.
Plain HTML, CSS, JavaScript	Provide a self-contained evidence UI without a frontend build system.
Prometheus / Grafana	Optional operational metrics and dashboards.
Docker Compose	Wire dependencies, persistent volumes, local ports, and credentials.
Terraform	Optional AWS storage and IAM foundation; it does not deploy the application platform.

The implementation uses small batch processing rather than a distributed processing cluster. It loads document/chunk rows into Python memory; its configured scale is at most 100 selected documents per crawl.

5. Website-to-release workflow

Initialize

`Pipeline.new_run()` creates a UUID-based run ID, stores selected nonsecret pipeline settings and their fingerprint, and captures the current active object's ETag. Airflow derives a stable pipeline ID from its DAG run ID so retries share a candidate. Selected configuration drift prevents resuming the same run. Not every operational setting is included in the fingerprint.

Scrape

Crawler reads robots rules, traverses same-origin links with a bounded queue, and selects product URLs matching the configured regular expression. It checks public destination addresses, rejects query-string URLs and unsafe paths, checks redirects, limits response size, and uses bounded retries. Rate limits and robots delays affect request pacing. DNS checks are defense in depth, not a replacement for a network egress policy.

The books adapter extracts title, rating, product-table fields such as price and availability, and description. Listing pages help discover products but are not selected documents. The generic adapter extracts a configured region after removing common active and navigation elements; it does not execute JavaScript.

The pipeline preserves response-body HTML bytes, extracted text, page metadata, robots information, errors, and selected-document rows. It fails if too few documents were collected.

Bronze

`DeltaTables.write()` writes extracted rows with an explicit document schema. This layer preserves the initial structured extraction and links back to raw HTML. Bronze is a Delta table, not simply the raw HTML directory.

Silver

The code normalizes Unicode and whitespace, strips most control characters, and redacts email patterns by default. It rejects missing titles, short content, invalid/stale/future timestamps, and selected instruction-like patterns. Rejections are recorded in quarantine.

DuckDB executes `sql/silver.sql`: partition by normalized content hash, select the deterministic winner ordered by URL and document ID, and order the output by document ID. Quality checks require enough documents, unique document IDs and content, an acceptable rejection fraction, and nonempty output. Reports are persisted before enforcement.

Gold

Documents become overlapping word windows: 220 words with 35-word overlap by default. Each chunk has a stable content-dependent ID, document ID, ordinal, URL, title, timestamp, raw key, and character offsets. Offsets index normalized Silver text, not HTML bytes. Tests check that slicing Silver text at those offsets exactly reproduces the chunk.

Gold validation checks chunk limits, uniqueness, document coverage, offsets, and text size. Chunking makes retrieval more focused and bounds the context sent to the model.

Embed and index

`OllamaEmbedder` resolves the embedding model's tag and digest. It adds `search_document`: when indexing and `search_query`: when embedding questions. These semantics are written for the default Nomic encoder; changing the model may require adapter changes, not just changing an environment variable.

The embedding cache key includes model identity, prefix version, and text hash. Unchanged text under the same model can reuse vectors across runs. Validation rejects wrong counts, inconsistent dimensions, nonfinite values, and all-zero vectors.

Chunks and vectors are upserted into `v1_<run_id>` in Chroma. The pipeline checks vector count and self-retrieval for up to five chunks. Self-retrieval verifies index integrity; it does not measure whether real questions will receive correct answers.

Publish

The pipeline creates a DuckDB metadata file and a release manifest containing exact table versions, quality reports, embedding identity, collection, counts, source, configuration fingerprint, and Git SHA. It rechecks model identity and collection count.

`Publications.publish()` validates required release evidence, prevents changing an existing manifest, records a publication intent, and conditionally updates the active pointer. A concurrent writer with an outdated ETag receives a conflict. The candidate's existence alone does not make it visible.

This is a controlled visibility protocol across multiple stores, not a distributed transaction covering every object and vector. Runtime checks cover selected invariants; they do not cryptographically verify every referenced artifact. Administrative mutation or deletion can still break a previously published release.

Retry and rollback

Completed stages return their stored results on retry. Failed stages can retry under the same configuration. An already committed active-pointer update can be acknowledged idempotently if a task failed before recording completion.

Rollback selects a retained validated release after checking its vector count and embedding model identity. It does not recreate deleted files, restore missing vectors, change models, or bypass dataset freshness checks.

6. Question-to-answer workflow

1. The UI sends `POST /ask` with the API key, question, and `top_k`.
2. FastAPI validates length and bounds, authenticates the key, and acquires the process-local request budget.
3. `RAGService.checked_release()` resolves the active release, checks encoder identity, and rejects a dataset older than the configured age using the run's creation timestamp.
4. Ollama embeds the question and Chroma searches that release's collection.
5. Results beyond cosine distance 0.65 are discarded by default. Evidence is capped at 3,500 characters per result and 15,000 characters overall.
6. The generator receives the question and evidence labeled `s1`, `s2`, etc., with a structured JSON answer schema and instructions to use evidence only. Default temperature is zero; thinking is disabled; output is bounded.
7. Validation requires inline source IDs to match cited IDs and each quote to be an exact substring of retrieved evidence.
8. The response contains an answer, source metadata, release ID, model names, and answered or abstained status. The request slot is released even on failure.

No relevant evidence, model abstention, and invalid citations produce distinct abstention reasons. Dependency errors produce HTTP 503. Exact quotes prove provenance, not that all generated claims logically follow from them. Pattern quarantine and prompting do not guarantee complete prompt-injection protection.

7. API and UI

Route	Purpose	Authentication
/	Evidence UI	Page itself is public locally
/static/*	Bundled UI assets	No API key
/docs	Interactive API schema	Protected operations still need a key
/healthz	Process liveness	No API key
/readyz	Release freshness, model identity, vector count, generation-model availability	No API key
/catalog	Active release metadata and quality checks	X-API-Key
GET /ask	Question via query parameters	X-API-Key
POST /ask	Question in JSON body; preferred for avoiding question-bearing URLs	X-API-Key
/metrics	Prometheus exposition	No API key; localhost boundary

Questions must be nonblank and at most 1,000 characters. `top_k` is 1–10. Defaults permit 30 accepted answer requests per minute and two concurrent requests for the single API process. HTTP 401 indicates authentication failure, 422 invalid input, 429 exhausted capacity, and 503 an unusable dependency or release.

The UI reads the key from its input field, keeps it in page memory, and displays answers and quotes using `textContent`. Source links are restricted to HTTP(S). Readiness is checked on page load and after catalog refresh; this is not a continuous background readiness monitor.

8. Complete application module map

Paths below are relative to `src/veritylake/`.

File	Responsibility
<code>__init__.py</code>	Package identity/version.
<code>config.py</code>	Environment settings, validation, selected public configuration and fingerprint.
<code>crawler.py</code>	URL policy, robots, rate pacing, retries, traversal, and extraction coordination.
<code>text.py</code>	HTML adapters, normalization, instruction patterns, and exact-offset chunking.
<code>quality.py</code>	Document preparation, quarantine reasons, Silver/Gold reports, and gate enforcement.
<code>storage.py</code>	Object-store abstraction, POSIX local adapter, S3 adapter, conditional writes, Delta options.

File	Responsibility
tables.py	Document/chunk schemas, Delta read/write, DuckDB deduplication and catalog construction.
sql/silver.sql	Deterministic content deduplication.
pipeline.py	Stage execution, results, provenance, candidate creation, publication, and rollback checks.
publication.py	Release validation, immutable-manifest convention, pointer selection, history, and switching.
embeddings.py	Ollama embedding API, model identity, validation, and cache.
vectors.py	Local/HTTP Chroma adapter, cosine collections, upserts, counts, and queries.
rag.py	Generation schema, prompt, release checks, retrieval, citations, and abstention.
api.py	FastAPI routes, authentication, request budgets, metrics, headers, and static serving.
cli.py	CLI commands for ingestion, stages, catalog, history, verification, rollback, and evaluation.
evaluation.py	Expected-URL retrieval recall and reciprocal-rank evaluation.
lineage.py	Durable OpenLineage START/COMPLETE/FAIL events and optional best-effort forwarding.
logging.py	JSON or readable logging; structured operational fields.
util.py	UTC timestamps, SHA-256, deterministic JSON encoding, strict run-ID validation.
ui/index.html	Key, question, retrieval controls, answer, and catalog page structure.
ui/app.js	API requests and safe rendering of evidence and release details.
ui/style.css	UI appearance and layout.

9. Persistent data and identities

raw/<run>/pages/	HTML, extracted text, page metadata
raw/<run>/documents.json	Selected extracted document rows
raw/<run>/robots.json	Captured crawl policy
raw/<run>/errors.json	Crawl diagnostics
bronze/<run>/documents/	Initial Delta snapshot
silver/<run>/documents/	Cleaned Delta snapshot
gold/<run>/chunks/	Chunk Delta snapshot
quarantine/<run>/silver.json	Rejection identities and reasons
quality/<run>/	Silver, Gold, embedding checks
embeddings/cache/	Model-aware vector cache
runs/<run>/run.json	Frozen selected configuration and base ETag
runs/<run>/stages/	Status, timing, outputs or failure type
lineage/<run>/<stage>/	Events grouped by stage attempt
catalog/runs/<run>/catalog.duckdb	Metadata SQL database
catalog/releases/<run>.json	Retained release manifests
catalog/publication_intents/	Intended publish/rollback operations
catalog/active.json	Current selected release
reports/<run>/benchmark.json	Pipeline timing

Document IDs hash canonical URLs. Raw content hashes describe response bytes; Silver content hashes describe normalized text. Chunk IDs incorporate document/content identity and chunking settings. Chroma holds chunk text as well as embeddings, which is why serving can retrieve evidence without reading Delta tables.

The catalog contains datasets, columns, and checks tables. It is a metadata database, not a copy of the complete document corpus. Named Docker volumes retain MinIO, Chroma, Ollama models, PostgreSQL, and Airflow configuration/logs across ordinary shutdowns.

10. Docker services and startup order

Service	Role
postgres	Airflow metadata database.
minio	Lake object storage; locally built source image.
minio-init	Create versioned bucket and separate writer/catalog-reader accounts.
chroma	Internal vector database with persistent data.
ollama	Internal local model server; CPU by default.
model-init	Wait for model/vector services, pull both models, warm the encoder.
airflow-init	Write local admin credentials and migrate Airflow metadata.
airflow-api	Airflow UI/API and execution API.
airflow-dag-processor	Discover and parse the DAG.
airflow-scheduler	Schedule tasks with LocalExecutor after dependencies initialize.
api	Evidence UI and RAG service using catalog-only S3 credentials.
tools	On-demand CLI container under the tools profile.
prometheus, grafana	Optional observability profile.

The app Dockerfile builds a Python 3.12 image, installs pinned direct dependencies, packages assets, and runs as a nonroot user. The Airflow image installs the same package and copies the DAG. The storage Dockerfile compiles MinIO and its client from specified source tags and includes source/license artifacts.

The DAG is `veritylake_scrape_to_rag`: `initialize` → `scrape` → `bronze` → `silver` → `gold` → `embed` → `publish`. It is unpaused with an `@once` schedule, one active run and task, two task retries, 30-second retry delay, 12-minute task timeout, and 20-minute DAG timeout. It will not automatically refresh every 72 hours; schedule changes or explicit triggers are needed.

11. Actual defaults

Setting	Default
Selected products / minimum accepted	60 / 50
Maximum visited URLs	180
Minimum request interval	0.5 seconds, increased by robots rules
Minimum document text	100 characters
Maximum rejected fraction	0.2
Dataset age limit	72 hours
Chunk words / overlap	220 / 35
Maximum chunks	1,500
Embedding model	<code>nomic-embed-text:v1.5</code>
Generation model	<code>qwen3:4b</code>
Embedding batch	16
Model request timeout	180 seconds
Maximum retrieval distance	0.65

The README previously included a second illustration naming smaller models; the settings and Compose defaults above determine actual behavior. The generation model tag is not pinned by application digest checks. `.env` configures Compose interpolation, but Compose passes an explicit subset to containers: adding an arbitrary Settings field to `.env` does not automatically forward it into every service.

12. Run locally

From a terminal:

```
cd /home/punam/Music/IMPFolder/Project/VerityLake_Senior_Portfolio/veritylake
python3 scripts/bootstrap.py
docker compose config --quiet
docker compose up --build -d
docker compose ps -a
docker compose logs --tail=100 model-init airflow-scheduler api
```

Bootstrap creates `.env` with unique secrets and mode 0600, and does not overwrite an existing file. Initial startup needs internet access for images, source compilation dependencies, Python dependencies, models, and the website. Generation and embeddings run locally after model installation. The full platform is not offline while scraping.

The repository suggests four CPU cores, 16 GB RAM, and 20 GB free disk as planning guidance. The inspected host has approximately 62 GiB total RAM and 329 GiB available workspace filesystem space. These are host observations, not measured application requirements.

Wait for actual publication, then verify:

```
curl --fail http://127.0.0.1:8000/readyz
python3 scripts/ask.py "What is the listed price of A Light in the Attic?"
python3 scripts/smoke.py
docker compose run --rm tools verify-release
```

Interface	Address	Local credential
Evidence UI	<code>http://localhost:8000</code>	API_KEY from <code>.env</code>
API docs	<code>http://localhost:8000/docs</code>	API key for protected requests
Airflow	<code>http://localhost:8080</code>	admin / AIRFLOW_ADMIN_PASSWORD
MinIO console	<code>http://localhost:9001</code>	MINIO_ROOT_USER / MINIO_ROOT_PASSWORD
Grafana, after make observe	<code>http://localhost:3000</code>	admin / GRAFANA_ADMIN_PASSWORD
Prometheus, after make observe	<code>http://localhost:9090</code>	Local interface

Read credentials locally in the editor. Chroma and Ollama have no published host ports. A healthy API process can still have a 503 readiness response until a valid release exists.

Common operations:

```
make trigger          # Another real Airflow run
make catalog          # Inspect selected release
make history           # Inspect retained manifests
make evaluate          # Small retrieval fixture set
make benchmark-airflow # Trigger and time an actual DAG run
make observe           # Start metrics and dashboards
docker compose down    # Stop/remove containers; keep volumes
```

For manual stage debugging, use `tools new-run` and pass the returned run ID to `make scrape`, `make etl`, `make embed`, and `make publish`. Run manual publication separately from active Airflow ingestion to avoid stale-writer conflicts. `make rollback RUN_ID=<retained-id>` switches to a retained candidate that passes rollback checks. Do not use `down --volumes` to perform an ordinary stop.

13. Scripts and supporting files

File/group	Purpose
scripts/bootstrap.py	Generate local configuration and separate credentials.
scripts/envutil.py	Parse local environment values for host helpers.
scripts/init_storage.py	Bucket, versioning, users, and policies.
scripts/init_models.py	Pull and warm models.
scripts/init_airflow.py	Authentication file and database migration.
scripts/ask.py	Send an authenticated sample question.
scripts/smoke.py	Check live health, readiness, catalog, and cited answer; save evidence.
scripts/check_static.py	Parse code/configuration and check selected isolation invariants.
scripts/check_dag.py	Validate the actual imported Airflow DAG.
scripts/benchmark_airflow.py	Trigger and observe real DAG duration.
scripts/backup.sh	Stop the stack and archive six persistent volumes with checksums; leaves it stopped.
scripts/restore.sh	Verify backup checksums and restore only to absent destination volumes.
scripts/deploy_api.sh	Deploy a digest-selected API image; return to previous image if readiness fails.
Makefile	Short wrappers for lifecycle, stages, checks, evaluation, and recovery.
compose.gpu.yaml	Optional GPU configuration overlay.
compose.ci.yaml	Loopback ports for real service integration tests.
requirements*.txt, pyproject.toml	Dependencies, package metadata, CLI entry point, test/lint configuration.
.dockerignore, .gitignore	Keep local/runtime artifacts out of image contexts and version control.
FILES.sha256	Original packaging checksums; later edited files naturally differ.
LICENSE, THIRD_PARTY_NOTICES.md	Original-code licensing and third-party boundaries.
CONTRIBUTING.md, SECURITY.md	Contribution workflow and threat/operating assumptions.

Backups include source data, database state, and models. Preserve `.env` separately; the script does not include it. The optional metrics/dashboard volumes are not in the six-volume backup list. A restore rehearsal must use a fresh project name and matching credentials.

14. Tests and evaluation

Test area	What it establishes
<code>test_config_storage.py</code>	Configuration bounds, keys, paths, run IDs, local conditional-write behavior.
<code>test_crawler.py</code>	URL restrictions, robots handling, retry pacing, redirect and response limits.
<code>test_text_quality.py</code>	Extraction, normalization, chunk offsets, and quality failures.
<code>test_embeddings_rag_api.py</code>	Embedding validation, caching, citations, abstention, auth and request limits.
<code>test_publication_pipeline.py</code>	Pipeline stages, retention, conflicts, retry, drift, and rollback behavior.
<code>test_adapter_contracts.py</code>	Mocked SDK/HTTP request shapes and CLI/evaluation/lineage contracts.
<code>integration/test_delta.py</code>	Real local Delta and DuckDB behavior when dependencies exist.
<code>integration/test_chroma.py</code>	Real persistent local Chroma adapter.
<code>integration/test_s3.py</code>	Boto adapter against Moto; not a real MinIO server.
<code>integration/test_airflow.py</code>	Real DAG import and graph when Airflow is installed.
<code>integration/test_service_stack.py</code>	Explicitly enabled actual MinIO/Delta/Chroma service contracts.

`tests/helpers.py` supplies fakes and `conftest.py` supplies fixtures. Passing fake-backed tests is useful but does not prove live model behavior. Existing packaging evidence reports 104 passes, five skipped integration modules, and roughly 83% coverage; see `reports/VALIDATION.md` for scope.

For a host development installation, use an isolated Python 3.12 or 3.13 environment, install `requirements-dev.txt` and the editable package, then run `make check`. Do not install these dependencies into unrelated projects' Python environments.

`eval/retrieval.jsonl` defines a small expected-URL set. Evaluation computes mean recall at k and reciprocal rank, deduplicating retrieved URLs for ranking. It evaluates raw retrieval, not the entire distance-filtered generation/citation path. `eval/adversarial.jsonl` and `eval/human_review.csv` support manual answer review. A five-minute warm-pipeline target is not a measured achievement until a real benchmark records it.

15. Monitoring, delivery, and cloud scope

The API exports HTTP counts and latency histograms, answer/abstention counts, the most recent readiness-probe result, dataset creation time, and active chunk count. Prometheus rules cover reachability, stale data, API errors, and slow answers. Grafana provisions the datasource and dashboard. `ops/loadtest.js` supplies a k6 workload. Pipeline stage duration is also stored in stage results and logs; not every operational fact is a Prometheus metric.

GitHub workflow responsibilities:

- `ci.yml`: Python 3.12/3.13 checks, dependency/secret scanning, actual storage/DAG contract jobs, and Terraform validation.
- `release.yml`: run CI, build/push four image variants, attach provenance/SBOM, scan the pushed digest, and sign it after scan success.

- `deploy.yml`: verify a signed API digest and deploy through SSH to a provisioned host using a configured GitHub environment.
- `live-acceptance.yml`: opt-in full-stack smoke, retrieval evaluation, warm pipeline target, and real Airflow benchmark.
- `.github/dependabot.yml`: dependency update configuration.

These files define workflows; they do not establish that GitHub has run them or that required deployment approvals are configured. The release workflow pushes before its scan, so publication to the registry and successful signed release are distinct steps. Deployment updates only the API image, not stateful services or Airflow.

Terraform defines a private, versioned, encrypted S3 bucket with TLS enforcement, a DynamoDB coordination table, reader/writer policies, and optional attachment to existing roles. Variables select region, bucket, environment, and role names; outputs expose provisioned resource identifiers. Role trust is managed separately. It does not provision compute, a network, Ollama, Airflow, Chroma, or an OIDC identity provider. Applying it would create billable cloud resources and is unnecessary for local startup.

16. Troubleshooting and limits

Symptom	Inspect next
Image build or pull fails	Exact tag availability, registry access, dependency pins, source build logs.
<code>model-init</code> fails	Ollama logs, download connectivity, model names, memory/disk.
UI loads but readiness is 503	Airflow stage state, active release, freshness, encoder identity, collection count.
HTTP 401	The UI key must match this project's <code>.env</code> .
HTTP 429	Wait for rate capacity or an active generation request to finish.
Crawl or Silver failure	<code>raw/<run>/errors.json</code> , robots report, quarantine and quality reports.
Publication conflict	Another run switched the pointer since this candidate began. Start a new run after inspecting history.
Abstention	Check retrieved evidence, distance cutoff, model output, and citation validity.
Model drift	Restore the matching encoder or build and publish a new index.
Dataset becomes stale	Trigger fresh ingestion; @once does not maintain freshness.

Current limits include one host, one shared API key, process-local request limits, no tenant isolation, no complete semantic answer verifier, heuristic instruction quarantine, and no reference-aware cleanup of retained releases. Delta/vector artifacts are immutable by application convention rather than storage-level object lock. Monitoring, backups, signing, and cloud templates need real deployment validation before production claims.

For deeper existing material, read `ARCHITECTURE.md` for invariants, `DECISIONS.md` for tradeoffs, `RUNBOOK.md` for operations, `DEVOPS.md` for delivery, `EVALUATION.md` for measurement, `PORTFOLIO.md` for demonstrations, and `SOURCES.md` for upstream references.

17. Suggested code reading order

Start with `config.py`, then `pipeline.py`. Follow its calls through `crawler.py`, `text.py`, `quality.py`, `tables.py`, `embeddings.py`, and `vectors.py`. Read `publication.py` with `storage.py` to understand safe visibility. Then read `rag.py`, `api.py`, and `ui/app.js` for serving. Finish with the DAG, Compose files, scripts, tests, and workflows. This follows actual data movement from source acquisition to the final answer.